Design patterns and backend architecture

Chapter 2: Design principles you should know

Practice designs, not claims about an employer.

Code examples are illustrative fragments, not complete runnable applications.

2. Design principles you should know

1. Single responsibility principle (SRP) [High]

Plain English: A component should have one coherent reason to change. This does not mean it must have only one method. A payment class that validates requests, calculates fees, saves records, and sends email may mix unrelated changes.

Professional answer: "I group behavior around one responsibility and keep boundaries clear. For example, payment policy, persistence, and notification change for different reasons, so I separate them when that separation makes changes and tests safer."

When to use and cost: Split when changes repeatedly touch unrelated logic or tests become difficult. Too many tiny classes can make a simple flow hard to follow. Ask: "What business decision belongs here, and who owns it?"

Demonstration — a change request: A `PaymentService` both computes fees and sends approval emails. Finance asks for a new fee rule; Support asks for a different email template. Editing and retesting the same service for unrelated requests indicates two reasons to change. Move fee decisions to `FeePolicy` and notification delivery to `NotificationSender`; keep `SubmitPayment` responsible for coordinating the use case.

Why SRP applies: The split follows separate business owners and change cadence, not a rule that every method needs a class. **Evidence:** A fee-policy unit test no longer needs a mail client; a notification failure does not change the fee calculation. If neither behavior changes separately, the split might be premature.

Code anchor (illustrative fragment):

```
final class SubmitPayment {
    private final FeePolicy fees;
    private final PaymentStore store;

    void accept(Payment p) {
        Money fee = fees.calculate(p);
        store.save(p, fee);
    }
}
// Fee policy and persistence change separately.
```

Verify it: Change the fee formula: only the fee-policy tests should change. A mail client should never be required for this unit test.

2. Open/closed principle (OCP) [High]

Plain English: Add a new variation with limited changes to stable, existing code. If every new payment method forces edits to one large switch, the design may need an extension point.

Professional answer: "I use an extension point when variations are real and growing. A PaymentMethod interface can let a new method be added without repeatedly changing the orchestration. I still avoid abstractions for speculative future features."

Trap: A change in business policy may legitimately change existing code. OCP is a design direction, not a ban on editing a class.

Demonstration — growing payment channels: Initially `submit()` contains `if (type == CARD) ... else if (type == ACH) ...`. When a third supported channel arrives, model `PaymentHandler` with implementations for each channel and register supported handlers. The stable `SubmissionService` asks a resolver for the matching handler and invokes `submit()`.

Why OCP applies: New channel *behavior* is added mainly through a new implementation, rather than editing the core processing algorithm. **Evidence:** Existing channel tests pass unchanged and a new channel gets contract tests. The resolver or its configuration may still change; OCP does not promise literally zero edits.

Code anchor (illustrative fragment):

```
interface PaymentHandler {
    PaymentRail rail();
    void submit(Payment payment);
}
final class HandlerResolver {
    private final Map<PaymentRail, PaymentHandler> handlers;
    PaymentHandler forRail(PaymentRail rail) {
        return Objects.requireNonNull(handlers.get(rail));
    }
}
// Add another PaymentHandler implementation for a new rail.
```

Verify it: Register an additional rail and run the same handler contract tests without editing submission orchestration. Registration still needs a supported-rail entry.

3. Liskov substitution principle (LSP) [High]

Plain English: If B is used wherever A is expected, B should honor A's promises. A subclass that throws for a supported parent operation or silently changes its meaning breaks that promise.

Professional answer: "I design subtypes around behavioral contracts, not only matching method signatures. A replacement should preserve the expectations callers depend on, including error behavior and business invariants."

Example: A read-only account should not extend a mutable account contract with `withdraw()` that always throws. Separate capabilities or use composition. Test the same contract against each implementation.

Demonstration — broken substitution: A `MutableAccount.withdraw(amount)` contract promises to withdraw a valid amount or return a documented insufficient-funds result. `ReadOnlyAccount extends MutableAccount` instead throws `UnsupportedOperationException` for every withdrawal. A caller that was correct for `MutableAccount` now fails unexpectedly.

Better design: Give `AccountView` a `balance()` operation; make `WithdrawableAccount` a separate capability. A read-only account implements only `AccountView`. **Why LSP applies:** Each implementation can honor every operation it advertises. **Evidence:** Run shared contract tests against all `WithdrawableAccount` implementations, including boundary/error behavior.

Code anchor (illustrative fragment):

```
interface AccountReader {
    Balance balance(AccountId id);
}
interface AccountWriter {
    Receipt withdraw(AccountId id, Money amount);
}
final class ReadOnlyAccount implements AccountReader {
    public Balance balance(AccountId id) { /* ... */ }
}
// Read-only accounts never promise withdraw().
```

Verify it: Run the same read-contract tests for every AccountReader. Do not fake support for withdrawals by throwing UnsupportedOperationException.

4. Interface segregation principle (ISP) [Medium]

Plain English: A caller should depend on only the operations it needs. One giant financial-platform interface makes unrelated clients know too much.

Professional answer: "I prefer focused interfaces based on consumers. A reconciliation reader should not need payment-submission or user-administration methods. This makes dependencies and permissions easier to reason about."

Cost: Too many one-method interfaces can create noise. Split where consumers genuinely need different contracts.

Demonstration — unnecessary authority: A report job only needs `listSettledPayments()`, but is given a `FinancialPlatform` interface containing `submitPayment()`, `deleteUser()`, and `updateLimit()`. Even if unused, those operations enlarge the apparent dependency and may complicate permissions and mocks.

Better design: Inject `SettlementReader` into the report job; give payment submission a separate `PaymentSubmitter` interface. **Why ISP applies:** Each consumer sees its own legitimate operations. **Evidence:** The reporting test needs only a read fake and the report job cannot call a write operation through its dependency.

Code anchor (illustrative fragment):

```
interface ReconciliationReader {
    List<Mismatch> findMismatches(BatchId batch);
}
interface PaymentSubmitter {
    Receipt submit(Payment payment);
}
final class ReportService {
    ReportService(ReconciliationReader reader) { /* ... */ }
}
// Reporting cannot accidentally submit payments.
```

Verify it: Compile the reporting module against only ReconciliationReader and confirm it cannot call payment submission methods.

5. Dependency inversion principle (DIP) [High]

Plain English: Business logic should depend on an appropriate contract rather than on a specific database, HTTP client, or vendor SDK. An adapter connects the contract to the outside system.

Professional answer: "I keep core business policy independent of infrastructure details. For example, a reconciliation service can depend on a `TransactionStore` contract while an adapter implements it with DB2 or another store. This allows focused tests and controlled replacement."

Clarify: Dependency injection supplies an implementation. Dependency inversion is the design of which component owns and depends on the interface. Constructor injection does not automatically make a design good.

Demonstration — a vendor replacement: Bad dependency: `ReconcileUseCase` imports `Db2JdbcClient` and contains DB2 SQL. Better dependency: it uses `TransactionStore.findUnmatched(batchId)`, a use-case-oriented contract; `Db2TransactionAdapter` owns SQL and row mapping. A test uses `InMemoryTransactionStore`.

Why DIP applies: The business-facing contract is controlled by the application, and the DB adapter depends on it—not the other way around. **Evidence:** The matching rule can be tested with no database; a storage migration changes the adapter but not reconciliation policy. Merely injecting `Db2JdbcClient` does **not** invert the dependency.

Code anchor (illustrative fragment):

```
interface PaymentStore {
    Optional<Payment> find(PaymentId id);
}
final class PaymentInquiry {
    private final PaymentStore store;
    PaymentInquiry(PaymentStore store) { this.store = store; }
}
final class JdbcPaymentStore implements PaymentStore {
    public Optional<Payment> find(PaymentId id) { /* SQL */ }
}
```

Verify it: Replace JdbcPaymentStore with a fake in a use-case test. The domain/application code does not import JDBC.

6. Cohesion and coupling [High]

Plain English: Cohesion means related work stays together. Coupling means components rely on each other's details. Aim for cohesive components with narrow, understandable connections.

Professional answer: "I evaluate a boundary by asking whether its logic belongs together and how many implementation details leak across it. Clear contracts and data ownership reduce the number of changes needed for a new feature."

Example: If changing a database column requires edits in five unrelated services, a storage detail has leaked through boundaries. Low coupling does not mean no collaboration: services still need contracts.

Demonstration — schema leakage: The API response, fee algorithm, and audit logger all read `PAYMENT_FEE_RULE_CODE` directly from a database row. Renaming the column forces edits across unrelated

modules. Introduce a repository mapping from the row to a domain concept such as `FeeRule`, then let other components consume the stable concept.

Why it fits: Mapping belongs with persistence (cohesion); the column name no longer leaks into pricing, transport, or audit (lower coupling). **Evidence:** A storage-column rename requires changing the repository mapping and its integration test, not the fee unit test or API contract.

Code anchor (illustrative fragment):

```
// Persistence maps database vocabulary to domain vocabulary.
final class PaymentRowMapper {
    Payment toDomain(PaymentRow row) {
        return new Payment(row.id(),
            FeeRule.parse(row.feeRuleCode()));
    }
}
// FeeCalculator accepts Payment, not PaymentRow.
```

Verify it: Rename fee_rule_code in the schema and verify that the API and fee calculator stay unchanged.

7. Composition over inheritance [High]

Plain English: Composition builds behavior using collaborating objects. Inheritance models a true is-a relationship. Composition often makes policy selection easier without a deep class tree.

Professional answer: "I use composition for variable behavior, such as fee calculation or provider integration. I reserve inheritance for a stable substitutable relationship. This keeps each policy independently testable."

Example: `PaymentProcessor` uses `FeePolicy`. It does not need subclasses for every market, channel, and currency combination. Those axes can combine without multiplying subclasses.

Demonstration — combinatorial variation: An inheritance plan creates `DomesticCardProcessor`, `InternationalCardProcessor`, `DomesticAchProcessor`, and `InternationalAchProcessor`. Adding an approval-policy variation multiplies subclasses again. Prefer `PaymentProcessor(FeePolicy, PaymentRail, ApprovalPolicy)` and select each collaborator explicitly.

Why it fits: Region, rail, and authorization are independent axes. Composition avoids pretending every combination is a distinct subtype. **Evidence:** Add a new approval policy and test it against existing rails without duplicating their code. Avoid composition when it produces arbitrary combinations that the business actually forbids; validate compatibility at assembly time.

Code anchor (illustrative fragment):

```
final class PaymentProcessor {
    private final FeePolicy fees;
    private final ApprovalPolicy approvals;
    private final PaymentRailGateway rail;

    PaymentProcessor(FeePolicy fees,
        ApprovalPolicy approvals, PaymentRailGateway rail) {
        this.fees = fees;
        this.approvals = approvals;
        this.rail = rail;
    }
}
```

Verify it: Change the approval policy without making new subclasses for every region and rail. Reject unsupported collaborator combinations at assembly time.

8. DRY, KISS, and YAGNI [High]

Plain English: DRY avoids repeating the same knowledge in many places. KISS favors the simplest design that meets the requirement. YAGNI says do not build a feature solely because it may be useful someday.

Professional answer: "I remove duplicated business rules, but I do not extract unrelated code just because two blocks look similar. I start with the simplest design that meets current behavior and add extension points when variation becomes real."

Trap: Two similar-looking transformations may follow different business policies. Forcing them into one shared helper can create many flags and make changes riskier. Duplicate syntax and duplicate knowledge are not always the same thing.

Demonstration — three different mistakes:

- **DRY:** Two services each hard-code the same legally required transaction rounding rule. Put the *single policy definition* in an owned domain component and test it once with boundary cases.
- **KISS:** A fixed two-case mapping is clearer as a small switch than a plugin framework with discovery, reflection, and configuration.
- **YAGNI:** Do not build a multi-region payment router before the business supports a second region or has a credible migration requirement.

Why these differ: DRY concerns duplicated *knowledge*, KISS concerns unnecessary complexity, and YAGNI concerns speculative requirements. **Evidence:** A real policy change needs one source update, the code remains readable, and no extension machinery exists without a current use case.

Code anchor (illustrative fragment):

```
// DRY: one owner for a real shared business rule.
Money roundFee(Money input) { /* agreed rule */ }

// KISS: for two stable options, a switch is enough.
boolean needsReview(RiskBand band) {
    return switch (band) {
        case HIGH -> true;
        case LOW -> false;
    };
}
// YAGNI: no plugin registry without a real second plugin.
```

Verify it: Test one rounding policy with boundary values. Avoid building extension machinery to replace a readable two-case switch.

9. Encapsulation and invariants [High]

Plain English: Encapsulation hides implementation details and protects rules that must always hold. An invariant is such a rule: an approved payment cannot have a negative amount, or an account change must be authorized.

Professional answer: "I keep state changes behind operations that enforce the business rules. I do not expose mutable internals and hope every caller validates them correctly. I also use database constraints for rules that must hold across concurrent application instances."

Example: A `Payment` may expose `approve()` with checks rather than a public `setStatus()`. Client-side validation and DTO validation do not replace domain or database protection.

Demonstration — illegal state: A publicly writable `payment.status` permits a caller to set `SETTLED` on a draft without authorization. Replace arbitrary setters with `authorize(actor)`, `markSettled(providerReference)`, and guarded transitions. Persist conditional updates to handle competing processors.

Why it fits: An invariant such as "settlement requires authorization and a provider reference" is enforced at the state-changing boundary. **Evidence:** Tests reject invalid transitions; a concurrent update test confirms only the expected prior version can advance. Object-level checks alone do not prevent database races; use version checks, locking, or constraints as appropriate.

Code anchor (illustrative fragment):

```
final class Payment {
    private Status status = Status.CREATED;
    void authorize() {
        if (status != Status.CREATED)
            throw new IllegalStateException("bad transition");
        status = Status.AUTHORIZED;
    }
    // No public setStatus(...).
}
```

Verify it: An attempt to authorize twice must fail. Add database concurrency protection when two processes can update the same row.

10. Design for testability and observability [High]

Plain English: A useful design makes correct behavior and failure behavior visible. Small contracts make tests easier; logs, metrics, and traces make production behavior explainable.

Professional answer: "I build seams around external dependencies and test business rules independently. I also make important state transitions and failure paths observable so an incident can be diagnosed from evidence rather than guesses."

Example: Test a retry rule with a fake provider, then integration-test real persistence and deduplication constraints. Track request ID, business ID, state, latency, and error category without logging secrets.

Demonstration — an intermittent timeout: Rather than performing a real provider call in a unit test, inject a `ClearingGateway` port whose fake returns success, a transient error, or an unknown outcome. Verify the correct business status for each case. In production, emit a trace with instruction ID, dependency name, latency, and a sanitized outcome category.

Why it fits: Tests prove the decision path, while telemetry helps explain the same path under live load. **Evidence:** A test reliably reproduces timeout behavior and an operator can correlate a pending instruction to its retries without exposing bank or identity secrets. Unit tests do not replace contract or integration tests.

Code anchor (illustrative fragment):

```
interface ClearingGateway {
    GatewayResult send(Payment p);
}
// In a unit test:
ClearingGateway timeout = p -> GatewayResult.UNKNOWN;
PaymentService service = new PaymentService(timeout);
assertEquals(Status.OUTCOME_UNKNOWN, service.submit(testPayment()));
```

Verify it: Inject a fake timeout and assert the business state. In production, log the instruction ID and sanitized error category, never account secrets.

11. Fail fast at boundaries, recover deliberately [Medium]

Plain English: Reject invalid input before doing expensive or irreversible work. For failures that may be temporary, recover only when the operation is safe to repeat.

Professional answer: "I validate preconditions early, return a clear error for invalid requests, and reserve retries for known transient failures. For a timed-out payment call, I first determine whether the operation completed before retrying."

Trap: "Fail fast" does not mean crash every service on a recoverable downstream problem. The boundary, recovery contract, and customer impact matter.

Demonstration — validation versus uncertainty: Reject a missing beneficiary or unsupported currency with a clear client error *before* storing an instruction. By contrast, if a remote provider times out after a valid submission, record `OUTCOME_UNKNOWN`; do not treat it as an invalid request or blindly re-send it.

Why it fits: Invalid input is deterministic; a remote timeout may conceal a successful side effect.
Evidence: An invalid-input test proves no instruction/event was persisted, while a timeout test proves the system schedules a status query or reconciliation. A retry is allowed only if the target's deduplication contract makes repetition safe.

Code anchor (illustrative fragment):

```
SubmitResult submit(Request request) {
    if (request.beneficiary() == null)
        throw new BadRequestException("beneficiary required");
    GatewayResult result = gateway.send(toPayment(request));
    if (result == GatewayResult.UNKNOWN)
        return SubmitResult.pendingReconciliation();
    return SubmitResult.from(result);
}
```

Verify it: The bad input path must make zero gateway calls. The unknown-outcome path must not blindly retry a potentially completed payment.

12. Separate policy from mechanism [Medium]

Plain English: Policy is the business decision; mechanism is how the system performs it. "Who may approve a payment?" is policy. "How do we read the role from a token?" is mechanism.

Professional answer: "I keep business decisions explicit and avoid burying them in transport or persistence code. A policy can then be reviewed by business owners and tested without making an HTTP call or using a real database."

Example: `ApprovalPolicy.canApprove(principal, payment)` should express the rule. A security adapter obtains the principal, and a repository loads the payment.

Demonstration — approval access: A controller inspects JWT claims and contains `if (role == MANAGER && amount < threshold) ...`. That mixes token parsing with a business approval rule. Instead, a security adapter resolves `Principal`, the application loads `Payment`, and `ApprovalPolicy.canApprove(principal, payment)` decides.

Why it fits: Business owners can review the approval policy independently of token format, and an identity-provider migration changes the adapter. **Evidence:** Table-driven tests vary role, tenant, limit, and payment state with no HTTP request. Authentication still belongs at the boundary; do not move token parsing into the domain.

Code anchor (illustrative fragment):

```
// Business rule: no JWT parsing here.
boolean canApprove(Principal actor, Payment p) {
    return actor.tenant().equals(p.tenant())
        && actor.approvalLimit().compareTo(p.amount()) >= 0;
}
// HTTP/JWT adapter builds Principal; policy uses it.
```

Verify it: Vary role, tenant, limit, and payment status in table-driven tests with no HTTP request.